Department of Computer Science and Engineering
Premier University

CSE 454: Compiler Construction Laboratory

Lab Report 06: Write a program to detect and remove left recursion from a given context-free grammar using C++

Submitted by:

Name	Mohammad Hafizur Rahman Sakib
ID	0222210005101118
Section	C
Session	Fall 2025 (8th Semester)
Date of performance	11.03.2026
Submission Date	31.03.2026

Submitted to:
Nazma Akther
Assistant Professor, Department of CSE
Premier University,Chattogram

Remarks

Experiment Name

Write a program to detect and remove left recursion from a given context-free grammar using C.

Objective

The objective of this experiment is to design and implement a C program that takes a context-free grammar as input, detects left recursion in the productions, and removes it by transforming the grammar into an equivalent non-left-recursive grammar. This helps in understanding the importance of eliminating left recursion before applying top-down parsing techniques in compiler design.

Algorithm

1. Start the program.
2. Read the number of grammar productions or read rules until an empty line.
3. For each production of the form $A \rightarrow \alpha$:
   (a) Extract the left-hand side (non-terminal) and right-hand side.
   (b) Split the right-hand side into individual productions using '—' as delimiter.
   (c) Separate the productions into two groups:
      - Those starting with the same non-terminal A (left recursive - α part).
      - Those not starting with A (β part).
   (d) If left recursive productions exist:
      - Print the original production and indicate left recursion is found.
      - Transform the grammar using the standard left recursion removal rule:

$$A \rightarrow \beta A' \quad | \quad \beta$$

$$A' \rightarrow \alpha A' \quad | \quad \epsilon$$

   (e) If no left recursion is found, print the production as it is.
4. Continue until all productions are processed.
5. Stop the program.

Source Code

```c
#include <stdio.h>
#include <string.h>

#define MAX 50
#define LEN 100

void removeLR(char g[MAX][LEN], int n) {
    for (int i = 0; i < n; i++) {
        char L[LEN], R[LEN], *p, prod[10][LEN], a[10][LEN], b[10][LEN];
        int pc = 0, ac = 0, bc = 0;

        sscanf(g[i], "%[^-]->%[^\n]", L, R);

        p = strtok(R, "|");
        while (p) { strcpy(prod[pc++], p); p = strtok(NULL, "|"); }

        for (int j = 0; j < pc; j++)
            if (!strncmp(prod[j], L, strlen(L)))
                strcpy(a[ac++], prod[j] + strlen(L));
            else
                strcpy(b[bc++], prod[j]);

        if (ac) {
            printf("\nLeft Recursion in %s\n%s -> ", L, L);
            for (int j = 0; j < bc; j++)
                printf("%s%s'%s", b[j], L, j < bc - 1 ? " | " : "");

            printf("\n%s' -> ", L);
            for (int j = 0; j < ac; j++)
                printf("%s%s'%s", a[j], L, j < ac - 1 ? " | " : "");

            printf(" | ε\n");
        } else {
            printf("\nNo left recursion: %s\n", g[i]);
        }
    }
}

int main() {
    char g[MAX][LEN];
    int n = 0;

    printf("Enter grammar (empty line to stop):\n");
    while (fgets(g[n], LEN, stdin) && g[n][0] != '\n') {
        g[n][strcspn(g[n], "\n")] = 0;
        n++;
    }

    removeLR(g, n);
    return 0;
}
```

Figure 1: Source code for detecting and removing left recursion from a given grammar

Output Screenshot

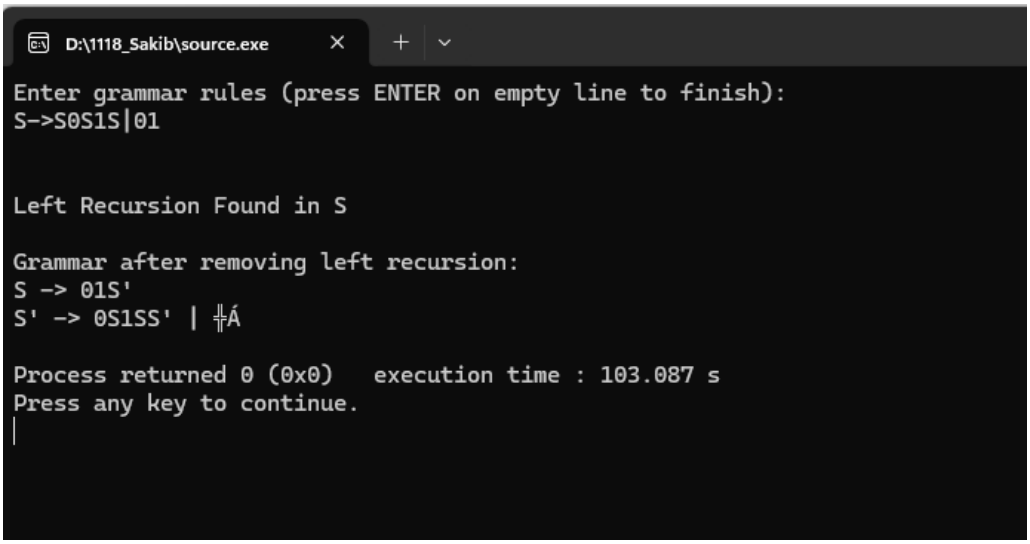

Figure 2: Output showing detection and removal of left recursion

Discussion

This experiment demonstrates the implementation of left recursion removal, which is a crucial step in making a grammar suitable for predictive (top-down) parsing. The program correctly identifies left-recursive productions by checking whether any alternative on the right-hand side begins with the same non-terminal as the left-hand side. It then applies the standard transformation to eliminate direct left recursion while preserving the language generated by the grammar.

The use of string tokenization with `strtok()` and careful parsing of productions makes the implementation efficient for laboratory purposes. This experiment provides hands-on understanding of grammar transformation techniques used in the syntax analysis phase of a compiler.